You want to convert the symbol to a one-call block like this:

```
{|x| x.capitalize }
```

As a first step, you can add a method to the Symbol class, which converts the symbol to a Proc object:

```
class Symbol
  def to_proc
    Proc.new {|x| x.send(self) }
  end
end
```

See how this method works? If you call it on, say, the :capitalize symbol, it returns a proc that takes an argument and calls capitalize on the argument. Now you can use to_proc and the & operator to convert a symbol to a Proc and then to a block:

```
names = ['bob', 'bill', 'heather']
names.map(&:capitalize.to_proc)   # => ["Bob", "Bill", "Heather"]
```

You can make this code even shorter. As it turns out, you can apply the & operator to any object, and it will take care of converting that object to a Proc by calling to_proc. (You didn't think we picked the name of the to_proc method randomly, did you?) So, you can simply write the following:

```
names = ['bob', 'bill', 'heather']
names.map(&:capitalize)   # => ["Bob", "Bill", "Heather"]
```

That's the trick known as *Symbol To Proc*. Neat, huh?

Spell: Symbol To Proc

The good news is that you don't have to write Symbol#to_proc, because it's already provided by Ruby. In fact, Ruby's implementation of Symbol#to_proc also supports blocks with more than one argument, which are required by methods such as inject:

```
# without Symbol#to_proc:
[1, 2, 5].inject(0) {|memo, obj| memo + obj }   # => 8

# with Symbol#to_proc:
[1, 2, 5].inject(0, &:+)  # => 8

# cool!
```